

# Employee Management System

## Mysql Wokrbench Code:

```
CREATE DATABASE employee_db;
```

```
USE employee_db;
```

```
CREATE TABLE department (  
    department_id INT PRIMARY KEY AUTO_INCREMENT,  
    department_name VARCHAR(100) UNIQUE NOT NULL  
);
```

```
INSERT INTO department (department_name) VALUES  
( 'HR' ),  
( 'Finance' ),  
( 'IT' ),  
( 'Sales' ),  
( 'Marketing' );
```

```
CREATE TABLE employee (  
    employee_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    gender VARCHAR(20) NOT NULL,  
    dob DATE NOT NULL,  
    contact VARCHAR(15) NOT NULL,  
    email VARCHAR(100),  
    address VARCHAR(200),  
    department_id INT NOT NULL,
```

```

        designation VARCHAR(100),

        joining_date DATE NOT NULL,

        basic_salary DOUBLE NOT NULL,

        allowances DOUBLE DEFAULT 0,

        deductions DOUBLE DEFAULT 0,

        employment_status VARCHAR(30) NOT NULL,


        FOREIGN KEY (department_id)

        REFERENCES department(department_id)

    );


select * from Employee;

```

## DBCONNECTION.JAVA

```

import java.sql.Connection;
import java.sql.DriverManager;

public class dbconnection {

    public static Connection getConnection() {

        Connection con = null;

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/employee_db",
                "root",
                "12345678"
            );

            System.out.println("Database connected successfully!");

        } catch (Exception e) {

            System.out.println("Database connection failed!");
            e.printStackTrace();

        }

    }
}

```

```

        return con;
    }
}

```

## EmployeeDAO.JAVA

```

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class EmployeeDAO {

    // CHECK WHETHER EMPLOYEE ID EXISTS

    public boolean employeeExists(int id) {

        String sql = "SELECT employee_id FROM employee WHERE employee_id = ?";

        try {

            Connection con = dbconnection.getConnection();

            if (con == null) {
                return false;
            }

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1, id);

            ResultSet rs = ps.executeQuery();

            return rs.next();

        } catch (Exception ex) {

            ex.printStackTrace();

        }

        return false;
    }

    // INSERT EMPLOYEE

    public boolean insertEmployee(Employee e) {

        String sql = "INSERT INTO employee VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    }
}

```

```

try {

    Connection con = dbconnection.getConnection();

    if (con == null) {
        return false;
    }

    PreparedStatement ps = con.prepareStatement(sql);

    ps.setInt(1, e.getEmployeeId());
    ps.setString(2, e.getName());
    ps.setString(3, e.getGender());
    ps.setString(4, e.getDob());
    ps.setString(5, e.getContact());
    ps.setString(6, e.getEmail());
    ps.setString(7, e.getAddress());
    ps.setInt(8, e.getDepartmentId());
    ps.setString(9, e.getDesignation());
    ps.setString(10, e.getJoiningDate());
    ps.setDouble(11, e.getBasicSalary());
    ps.setDouble(12, e.getAllowances());
    ps.setDouble(13, e.getDeductions());
    ps.setString(14, e.getEmploymentStatus());

    int rows = ps.executeUpdate();

    if (rows > 0) {
        return true;
    }

} catch (Exception ex) {

    ex.printStackTrace();

}

return false;
}

// UPDATE EMPLOYEE

public boolean updateEmployee(Employee e) {

    String sql = "UPDATE employee SET " +
        "name=?, gender=?, dob=?, contact=?, email=?, address=?, " +
        "department_id=?, designation=?, joining_date=?, basic_salary=?, " +
        "allowances=?, deductions=?, employment_status=? " +
        "WHERE employee_id=?";

    try {

        Connection con = dbconnection.getConnection();

        if (con == null) {
            return false;

```

```

    }

    PreparedStatement ps = con.prepareStatement(sql);

    ps.setString(1, e.getName());
    ps.setString(2, e.getGender());
    ps.setString(3, e.getDob());
    ps.setString(4, e.getContact());
    ps.setString(5, e.getEmail());
    ps.setString(6, e.getAddress());
    ps.setInt(7, e.getDepartmentId());
    ps.setString(8, e.getDesignation());
    ps.setString(9, e.getJoiningDate());
    ps.setDouble(10, e.getBasicSalary());
    ps.setDouble(11, e.getAllowances());
    ps.setDouble(12, e.getDeductions());
    ps.setString(13, e.getEmploymentStatus());
    ps.setInt(14, e.getEmployeeId());

    int rows = ps.executeUpdate();

    if (rows > 0) {
        return true;
    }

} catch (Exception ex) {

    ex.printStackTrace();

}

return false;
}

// SEARCH EMPLOYEE BY ID

public Employee searchEmployee(int id) {

    String sql = "SELECT * FROM employee WHERE employee_id = ?";

    try {

        Connection con = dbconnection.getConnection();

        if (con == null) {
            return null;
        }

        PreparedStatement ps = con.prepareStatement(sql);

        ps.setInt(1, id);

        ResultSet rs = ps.executeQuery();

        if (rs.next()) {

```

```

        Employee e = new Employee();

        e.setEmployeeId(rs.getInt("employee_id"));
        e.setName(rs.getString("name"));
        e.setGender(rs.getString("gender"));
        e.setDob(rs.getString("dob"));
        e.setContact(rs.getString("contact"));
        e.setEmail(rs.getString("email"));
        e.setAddress(rs.getString("address"));
        e.setDepartmentId(rs.getInt("department_id"));
        e.setDesignation(rs.getString("designation"));
        e.setJoiningDate(rs.getString("joining_date"));
        e.setBasicSalary(rs.getDouble("basic_salary"));
        e.setAllowances(rs.getDouble("allowances"));
        e.setDeductions(rs.getDouble("deductions"));
        e.setEmploymentStatus(rs.getString("employment_status"));

        return e;
    }

    } catch (Exception ex) {

        ex.printStackTrace();

    }

    return null;
}

// SEARCH BY DEPARTMENT

public String searchByDepartment(String department) {

    String sql = "SELECT e.employee_id, e.name, d.department_name, " +
        "e.designation, e.employment_status " +
        "FROM employee e " +
        "JOIN department d ON e.department_id = d.department_id " +
        "WHERE d.department_name = ?";

    return getSearchResult(sql, department);
}

// SEARCH BY DESIGNATION

public String searchByDesignation(String designation) {

    String sql = "SELECT e.employee_id, e.name, d.department_name, " +
        "e.designation, e.employment_status " +
        "FROM employee e " +
        "JOIN department d ON e.department_id = d.department_id " +
        "WHERE e.designation = ?";

    return getSearchResult(sql, designation);
}

```

```
// SEARCH BY EMPLOYMENT STATUS
```

```
public String searchByStatus(String status) {

    String sql = "SELECT e.employee_id, e.name, d.department_name, " +
        "e.designation, e.employment_status " +
        "FROM employee e " +
        "JOIN department d ON e.department_id = d.department_id " +
        "WHERE e.employment_status = ?";

    return getSearchResult(sql, status);
}
```

```
// COMMON SEARCH METHOD
```

```
private String getSearchResult(String sql, String value) {

    StringBuilder result = new StringBuilder();

    try {

        Connection con = dbconnection.getConnection();

        if (con == null) {
            return "Database connection failed";
        }

        PreparedStatement ps = con.prepareStatement(sql);

        ps.setString(1, value);

        ResultSet rs = ps.executeQuery();

        boolean found = false;

        while (rs.next()) {

            found = true;

            result.append("ID: ")
                .append(rs.getInt("employee_id"))
                .append("    Name: ")
                .append(rs.getString("name"))
                .append("    Department: ")
                .append(rs.getString("department_name"))
                .append("    Designation: ")
                .append(rs.getString("designation"))
                .append("    Status: ")
                .append(rs.getString("employment_status"))
                .append("\n");

        }

        if (!found) {
            return "No employees found";
        }

    }

}
```

```

    } catch (Exception ex) {

        ex.printStackTrace();
        return "Error while searching";

    }

    return result.toString();
}

// DELETE EMPLOYEE

public boolean deleteEmployee(int id) {

    String sql = "DELETE FROM employee WHERE employee_id = ?";

    try {

        Connection con = dbconnection.getConnection();

        if (con == null) {
            return false;
        }

        PreparedStatement ps = con.prepareStatement(sql);

        ps.setInt(1, id);

        int rows = ps.executeUpdate();

        if (rows > 0) {
            return true;
        }

    } catch (Exception ex) {

        ex.printStackTrace();

    }

    return false;
}

// VIEW ALL EMPLOYEES

public String viewAllEmployees() {

    String sql = "SELECT e.*, d.department_name " +
        "FROM employee e " +
        "JOIN department d ON e.department_id = d.department_id";

    StringBuilder result = new StringBuilder();

    try {

```

```

        Connection con = dbconnection.getConnection();

        if (con == null) {
            return "Database connection failed";
        }

        PreparedStatement ps = con.prepareStatement(sql);

        ResultSet rs = ps.executeQuery();

        boolean found = false;

        while (rs.next()) {

            found = true;

            result.append("ID: ")
                .append(rs.getInt("employee_id"))
                .append("    Name: ")
                .append(rs.getString("name"))
                .append("    Department: ")
                .append(rs.getString("department_name"))
                .append("    Designation: ")
                .append(rs.getString("designation"))
                .append("    Status: ")
                .append(rs.getString("employment_status"))
                .append("    Basic Salary: ")
                .append(rs.getDouble("basic_salary"))
                .append("\n");

        }

        if (!found) {
            return "No employees found";
        }

    } catch (Exception ex) {

        ex.printStackTrace();
        return "Error while retrieving employees";

    }

    return result.toString();
}
}

```

## Employee.JAVA

```

public class Employee {

    private int employeeId;
    private String name;

```

```

private String gender;
private String dob;
private String contact;
private String email;
private String address;
private int departmentId;
private String designation;
private String joiningDate;
private double basicSalary;
private double allowances;
private double deductions;
private String employmentStatus;

public Employee() {

}

public Employee(int employeeId, String name, String gender,
                String dob, String contact, String email,
                String address, int departmentId,
                String designation, String joiningDate,
                double basicSalary, double allowances,
                double deductions, String employmentStatus) {

    this.employeeId = employeeId;
    this.name = name;
    this.gender = gender;
    this.dob = dob;
    this.contact = contact;
    this.email = email;
    this.address = address;
    this.departmentId = departmentId;
    this.designation = designation;
    this.joiningDate = joiningDate;
    this.basicSalary = basicSalary;
    this.allowances = allowances;
    this.deductions = deductions;
    this.employmentStatus = employmentStatus;
}

public int getEmployeeId() {
    return employeeId;
}

public void setEmployeeId(int employeeId) {
    this.employeeId = employeeId;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getGender() {

```

```
        return gender;
    }

    public void setGender(String gender) {
        this.gender = gender;
    }

    public String getDob() {
        return dob;
    }

    public void setDob(String dob) {
        this.dob = dob;
    }

    public String getContact() {
        return contact;
    }

    public void setContact(String contact) {
        this.contact = contact;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public int getDepartmentId() {
        return departmentId;
    }

    public void setDepartmentId(int departmentId) {
        this.departmentId = departmentId;
    }

    public String getDesignation() {
        return designation;
    }

    public void setDesignation(String designation) {
        this.designation = designation;
    }

    public String getJoiningDate() {
        return joiningDate;
    }
}
```

```

    }

    public void setJoiningDate(String joiningDate) {
        this.joiningDate = joiningDate;
    }

    public double getBasicSalary() {
        return basicSalary;
    }

    public void setBasicSalary(double basicSalary) {
        this.basicSalary = basicSalary;
    }

    public double getAllowances() {
        return allowances;
    }

    public void setAllowances(double allowances) {
        this.allowances = allowances;
    }

    public double getDeductions() {
        return deductions;
    }

    public void setDeductions(double deductions) {
        this.deductions = deductions;
    }

    public String getEmploymentStatus() {
        return employmentStatus;
    }

    public void setEmploymentStatus(String employmentStatus) {
        this.employmentStatus = employmentStatus;
    }
}

```

## Payroll.JAVA

```

public class Payroll {

    public static double calculateGrossSalary(
        double basicSalary,
        double allowances) {

        return basicSalary + allowances;
    }

    public static double calculateNetSalary(
        double basicSalary,
        double allowances,
        double deductions) {

```

```

        double grossSalary =
            basicSalary + allowances;

        return grossSalary - deductions;
    }
}

```

## EmployeeCRUD.JAVA

```

import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;

public class EmployeeCRUD extends Frame {

    Label lblTitle;
    Label lblId, lblName, lblGender, lblDob;
    Label lblContact, lblEmail, lblAddress, lblDept;
    Label lblDesignation, lblJoining, lblBasic;
    Label lblAllowances, lblDeductions, lblStatus;
    Label lblSearch;

    TextField txtId, txtName, txtDob;
    TextField txtContact, txtEmail, txtAddress;
    TextField txtDesignation, txtJoining;
    TextField txtBasic, txtAllowances, txtDeductions;
    TextField txtSearch;

    Choice genderChoice;
    Choice departmentChoice;
    Choice statusChoice;
    Choice searchChoice;

    Button btnAdd, btnSearch, btnUpdate;
    Button btnDelete, btnClear, btnView;
    Button btnPayroll;

    TextArea txtArea;

    EmployeeDAO dao = new EmployeeDAO();

    public EmployeeCRUD() {

        setTitle("Employee Management and Payroll System");

        setSize(1000, 750);

        setLayout(new BorderLayout(10, 10));
    }
}

```

```

// TITLE

lblTitle = new Label(
    "EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM",
    Label.CENTER
);

lblTitle.setFont(
    new Font("Arial", Font.BOLD, 22)
);

add(lblTitle, BorderLayout.NORTH);

// FORM PANEL

Panel formPanel = new Panel();

formPanel.setLayout(
    new GridLayout(7, 4, 10, 10)
);

lblId = new Label("Employee ID *");
txtId = new TextField();

lblName = new Label("Name *");
txtName = new TextField();

lblGender = new Label("Gender *");

genderChoice = new Choice();

genderChoice.add("Male");
genderChoice.add("Female");
genderChoice.add("Other");

lblDob = new Label("Date of Birth *");
txtDob = new TextField();

lblContact = new Label("Contact Number *");
txtContact = new TextField();

lblEmail = new Label("Email");
txtEmail = new TextField();

lblAddress = new Label("Address");
txtAddress = new TextField();

lblDept = new Label("Department *");

departmentChoice = new Choice();

departmentChoice.add("HR");
departmentChoice.add("Finance");
departmentChoice.add("IT");
departmentChoice.add("Sales");
departmentChoice.add("Marketing");

```

```
lblDesignation = new Label("Designation *");
txtDesignation = new TextField();

lblJoining = new Label("Date of Joining *");
txtJoining = new TextField();

lblBasic = new Label("Basic Salary *");
txtBasic = new TextField();

lblAllowances = new Label("Allowances");
txtAllowances = new TextField();

lblDeductions = new Label("Deductions");
txtDeductions = new TextField();

lblStatus = new Label("Employment Status *");

statusChoice = new Choice();

statusChoice.add("Active");
statusChoice.add("Inactive");
statusChoice.add("On Leave");

formPanel.add(lblId);
formPanel.add(txtId);

formPanel.add(lblName);
formPanel.add(txtName);

formPanel.add(lblGender);
formPanel.add(genderChoice);

formPanel.add(lblDob);
formPanel.add(txtDob);

formPanel.add(lblContact);
formPanel.add(txtContact);

formPanel.add(lblEmail);
formPanel.add(txtEmail);

formPanel.add(lblAddress);
formPanel.add(txtAddress);

formPanel.add(lblDept);
formPanel.add(departmentChoice);

formPanel.add(lblDesignation);
formPanel.add(txtDesignation);

formPanel.add(lblJoining);
formPanel.add(txtJoining);

formPanel.add(lblBasic);
formPanel.add(txtBasic);
```

```

formPanel.add(lblAllowances);
formPanel.add(txtAllowances);

formPanel.add(lblDeductions);
formPanel.add(txtDeductions);

formPanel.add(lblStatus);
formPanel.add(statusChoice);

// SEARCH PANEL

Panel searchPanel = new Panel();

searchPanel.setLayout(
    new FlowLayout()
);

lblSearch = new Label("Search By:");

searchChoice = new Choice();

searchChoice.add("Employee ID");
searchChoice.add("Department");
searchChoice.add("Designation");
searchChoice.add("Employment Status");

txtSearch = new TextField(15);

searchPanel.add(lblSearch);
searchPanel.add(searchChoice);
searchPanel.add(txtSearch);

// BUTTON PANEL

Panel buttonPanel = new Panel();

buttonPanel.setLayout(
    new FlowLayout()
);

btnAdd = new Button("Add");
btnSearch = new Button("Search");
btnUpdate = new Button("Update");
btnDelete = new Button("Delete");
btnClear = new Button("Clear");
btnView = new Button("View All");
btnPayroll = new Button("Payroll");

buttonPanel.add(btnAdd);
buttonPanel.add(btnSearch);
buttonPanel.add(btnUpdate);
buttonPanel.add(btnDelete);
buttonPanel.add(btnClear);
buttonPanel.add(btnView);
buttonPanel.add(btnPayroll);

Panel centerPanel = new Panel();

centerPanel.setLayout(
    new BorderLayout(10, 10)

```

```

);

centerPanel.add(
    formPanel,
    BorderLayout.CENTER
);

Panel bottomPanel = new Panel();

bottomPanel.setLayout(
    new BorderLayout()
);

bottomPanel.add(
    searchPanel,
    BorderLayout.NORTH
);

bottomPanel.add(
    buttonPanel,
    BorderLayout.SOUTH
);

centerPanel.add(
    bottomPanel,
    BorderLayout.SOUTH
);

add(
    centerPanel,
    BorderLayout.CENTER
);

// OUTPUT AREA

txtArea = new TextArea();

txtArea.setEditable(false);

add(
    txtArea,
    BorderLayout.SOUTH
);

// CLOSE WINDOW

addWindowListener(new WindowAdapter() {

    @Override
    public void windowClosing(WindowEvent e) {

        dispose();

    }

});

// =====
// ADD BUTTON
// =====

```

```

btnAdd.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {

        try {

            if (!validateFields()) {
                return;
            }

            int id =
                Integer.parseInt(
                    txtId.getText()
                );

            if (dao.employeeExists(id)) {

                txtArea.setText(
                    "Employee ID already exists"
                );

                return;
            }

            Employee emp =
                createEmployeeFromFields();

            boolean result =
                dao.insertEmployee(emp);

            if (result) {

                txtArea.setText(
                    "Employee Added Successfully"
                );

            } else {

                txtArea.setText(
                    "Failed to Add Employee"
                );

            }

        } catch (NumberFormatException ex) {

            txtArea.setText(
                "Employee ID and salary must be numeric"
            );

        } catch (Exception ex) {

            txtArea.setText(
                "Error while adding employee"
            );

            ex.printStackTrace();
        }
    }
});

```

```

    }

}

});

// =====
// UPDATE BUTTON
// =====

btnUpdate.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {

        try {

            if (!validateFields()) {
                return;
            }

            int id =
                Integer.parseInt(
                    txtId.getText()
                );

            if (!dao.employeeExists(id)) {

                txtArea.setText(
                    "Employee ID Not Found"
                );

                return;
            }

            Employee emp =
                createEmployeeFromFields();

            boolean result =
                dao.updateEmployee(emp);

            if (result) {

                txtArea.setText(
                    "Employee Updated Successfully"
                );

            } else {

                txtArea.setText(
                    "Failed to Update Employee"
                );

            }

        } catch (NumberFormatException ex) {

            txtArea.setText(

```

```

        "Employee ID and salary must be numeric"
    );
} catch (Exception ex) {

    txtArea.setText(
        "Error while updating employee"
    );

    ex.printStackTrace();

}

}

});

// =====
// SEARCH BUTTON
// =====

btnSearch.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {

        try {

            String searchType =
                searchChoice.getSelectedItem();

            String value =
                txtSearch.getText().trim();

            if (value.isEmpty()) {

                txtArea.setText(
                    "Please enter a search value"
                );

                return;
            }

            // SEARCH BY ID

            if (searchType.equals("Employee ID")) {

                int id =
                    Integer.parseInt(value);

                Employee emp =
                    dao.searchEmployee(id);

                if (emp != null) {

                    fillEmployeeFields(emp);

                    txtArea.setText(
                        "Employee Found Successfully"
                    )
                }
            }
        }
    }
});

```

```

        );

    } else {

        txtArea.setText(
            "Employee ID Not Found"
        );

    }

}

// SEARCH BY DEPARTMENT

else if (
    searchType.equals("Department")
) {

    String result =
        dao.searchByDepartment(value);

    txtArea.setText(result);

}

// SEARCH BY DESIGNATION

else if (
    searchType.equals("Designation")
) {

    String result =
        dao.searchByDesignation(value);

    txtArea.setText(result);

}

// SEARCH BY STATUS

else if (
    searchType.equals(
        "Employment Status"
    )
) {

    String result =
        dao.searchByStatus(value);

    txtArea.setText(result);

}

} catch (NumberFormatException ex) {

    txtArea.setText(
        "Employee ID must be numeric"
    );

} catch (Exception ex) {

```

```

        txtArea.setText(
            "Error while searching"
        );

        ex.printStackTrace();

    }

}

});

// =====
// DELETE BUTTON
// =====

btnDelete.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {

        try {

            if (txtId.getText().trim().isEmpty()) {

                txtArea.setText(
                    "Enter Employee ID"
                );

                return;
            }

            int id =
                Integer.parseInt(
                    txtId.getText()
                );

            if (!dao.employeeExists(id)) {

                txtArea.setText(
                    "Employee ID Not Found"
                );

                return;
            }

            boolean result =
                dao.deleteEmployee(id);

            if (result) {

                txtArea.setText(
                    "Employee Deleted Successfully"
                );

                clearFields();

            } else {

```

```

        txtArea.setText(
            "Failed to Delete Employee"
        );
    }

} catch (NumberFormatException ex) {

    txtArea.setText(
        "Employee ID must be numeric"
    );

} catch (Exception ex) {

    txtArea.setText(
        "Error while deleting employee"
    );

    ex.printStackTrace();

}

}

});

// =====
// CLEAR BUTTON
// =====

btnClear.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {

        clearFields();

        txtSearch.setText("");

        txtArea.setText("");

    }

});

// =====
// VIEW ALL BUTTON
// =====

btnView.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {

        String result =
            dao.viewAllEmployees();

        txtArea.setText(result);

    }

});

```

```

// =====
// PAYROLL BUTTON
// =====

btnPayroll.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {

        try {

            if (txtBasic.getText().trim().isEmpty()) {

                txtArea.setText(
                    "Enter Basic Salary"
                );

                return;
            }

            double basic =
                Double.parseDouble(
                    txtBasic.getText()
                );

            double allowances = 0;

            if (!txtAllowances.getText()
                .trim().isEmpty()) {

                allowances =
                    Double.parseDouble(
                        txtAllowances.getText()
                    );
            }

            double deductions = 0;

            if (!txtDeductions.getText()
                .trim().isEmpty()) {

                deductions =
                    Double.parseDouble(
                        txtDeductions.getText()
                    );
            }

            if (basic < 0 ||
                allowances < 0 ||
                deductions < 0) {

                txtArea.setText(
                    "Salary values cannot be negative"
                );

                return;
            }

```

```

double gross =
    Payroll.calculateGrossSalary(
        basic,
        allowances
    );

double net =
    Payroll.calculateNetSalary(
        basic,
        allowances,
        deductions
    );

if (deductions > gross) {

    txtArea.setText(
        "Deductions cannot exceed Gross Salary"
    );

    return;
}

txtArea.setText(
    "PAYROLL DETAILS\n\n" +

        "Basic Salary : " +
        basic +

        "\nAllowances : " +
        allowances +

        "\nDeductions : " +
        deductions +

        "\nGross Salary : " +
        gross +

        "\nNet Salary : " +
        net
    );

} catch (NumberFormatException ex) {

    txtArea.setText(
        "Please enter valid salary values"
    );

}

}

});

}

// =====
// CREATE EMPLOYEE OBJECT
// =====

```

```

private Employee createEmployeeFromFields() {

    int id =
        Integer.parseInt(
            txtId.getText()
        );

    String name =
        txtName.getText().trim();

    String gender =
        genderChoice.getSelectedItem();

    String dob =
        txtDob.getText().trim();

    String contact =
        txtContact.getText().trim();

    String email =
        txtEmail.getText().trim();

    String address =
        txtAddress.getText().trim();

    int departmentId =
        getDepartmentId(
            departmentChoice.getSelectedItem()
        );

    String designation =
        txtDesignation.getText().trim();

    String joining =
        txtJoining.getText().trim();

    double basic =
        Double.parseDouble(
            txtBasic.getText()
        );

    double allowances = 0;

    if (!txtAllowances.getText()
        .trim().isEmpty()) {

        allowances =
            Double.parseDouble(
                txtAllowances.getText()
            );
    }
}

```

```

double deductions = 0;

if (!txtDeductions.getText()
    .trim().isEmpty()) {

    deductions =
        Double.parseDouble(
            txtDeductions.getText()
        );
}

String status =
    statusChoice.getSelectedItem();

return new Employee(
    id,
    name,
    gender,
    dob,
    contact,
    email,
    address,
    departmentId,
    designation,
    joining,
    basic,
    allowances,
    deductions,
    status
);
}

// =====
// VALIDATE FIELDS
// =====

private boolean validateFields() {

    if (txtId.getText().trim().isEmpty() ||
        txtName.getText().trim().isEmpty() ||
        txtDob.getText().trim().isEmpty() ||
        txtContact.getText().trim().isEmpty() ||
        txtDesignation.getText().trim().isEmpty() ||
        txtJoining.getText().trim().isEmpty() ||
        txtBasic.getText().trim().isEmpty()) {

        txtArea.setText(
            "Please fill all mandatory fields"
        );

        return false;
    }
}

// VALIDATE ID

try {

    int id =
        Integer.parseInt(
            txtId.getText()

```

```

        );

        if (id <= 0) {

            txtArea.setText(
                "Employee ID must be positive"
            );

            return false;
        }

    } catch (NumberFormatException ex) {

        txtArea.setText(
            "Employee ID must be numeric"
        );

        return false;
    }

}

// VALIDATE DATES

if (!isValidDate(txtDob.getText().trim())) {

    txtArea.setText(
        "Date of Birth must be YYYY-MM-DD"
    );

    return false;
}

if (!isValidDate(txtJoining.getText().trim())) {

    txtArea.setText(
        "Date of Joining must be YYYY-MM-DD"
    );

    return false;
}

// VALIDATE SALARY

try {

    double basic =
        Double.parseDouble(
            txtBasic.getText()
        );

    double allowances = 0;

    if (!txtAllowances.getText()
        .trim().isEmpty()) {

        allowances =
            Double.parseDouble(
                txtAllowances.getText()
            );
    }

}

```

```

        double deductions = 0;

        if (!txtDeductions.getText()
            .trim().isEmpty()) {

            deductions =
                Double.parseDouble(
                    txtDeductions.getText()
                );
        }

        if (basic < 0 ||
            allowances < 0 ||
            deductions < 0) {

            txtArea.setText(
                "Salary values cannot be negative"
            );

            return false;
        }

        double gross =
            basic + allowances;

        if (deductions > gross) {

            txtArea.setText(
                "Deductions cannot exceed Gross Salary"
            );

            return false;
        }
    } catch (NumberFormatException ex) {

        txtArea.setText(
            "Salary values must be numeric"
        );

        return false;
    }

    return true;
}

// =====
// DATE VALIDATION
// =====

private boolean isValidDate(String date) {

    try {

        LocalDate.parse(date);

        return true;
    }

```

```

        } catch (DateTimeParseException ex) {

            return false;
        }
    }

// =====
// FILL EMPLOYEE DETAILS
// =====

private void fillEmployeeFields(Employee emp) {

    txtId.setText(
        String.valueOf(
            emp.getEmployeeId()
        )
    );

    txtName.setText(
        emp.getName()
    );

    genderChoice.select(
        emp.getGender()
    );

    txtDob.setText(
        emp.getDob()
    );

    txtContact.setText(
        emp.getContact()
    );

    txtEmail.setText(
        emp.getEmail()
    );

    txtAddress.setText(
        emp.getAddress()
    );

    setDepartment(
        emp.getDepartmentId()
    );

    txtDesignation.setText(
        emp.getDesignation()
    );

    txtJoining.setText(
        emp.getJoiningDate()
    );

    txtBasic.setText(
        String.valueOf(
            emp.getBasicSalary()
        )
    );

    txtAllowances.setText(
        String.valueOf(
            emp.getAllowances()
        )
    );
}

```

```

        )
    );

    txtDeductions.setText(
        String.valueOf(
            emp.getDeductions()
        )
    );

    statusChoice.select(
        emp.getEmploymentStatus()
    );
}

// =====
// DEPARTMENT ID
// =====

private int getDepartmentId(
    String department) {

    if (department.equals("HR"))
        return 1;

    if (department.equals("Finance"))
        return 2;

    if (department.equals("IT"))
        return 3;

    if (department.equals("Sales"))
        return 4;

    if (department.equals("Marketing"))
        return 5;

    return 1;
}

// =====
// SET DEPARTMENT
// =====

private void setDepartment(
    int departmentId) {

    if (departmentId == 1)
        departmentChoice.select("HR");

    else if (departmentId == 2)
        departmentChoice.select("Finance");

    else if (departmentId == 3)
        departmentChoice.select("IT");

    else if (departmentId == 4)
        departmentChoice.select("Sales");

    else if (departmentId == 5)
        departmentChoice.select("Marketing");
}

```

```

// =====
// CLEAR FIELDS
// =====

private void clearFields() {

    txtId.setText("");
    txtName.setText("");
    txtDob.setText("");
    txtContact.setText("");
    txtEmail.setText("");
    txtAddress.setText("");
    txtDesignation.setText("");
    txtJoining.setText("");
    txtBasic.setText("");
    txtAllowances.setText("");
    txtDeductions.setText("");

    genderChoice.select("Male");
    departmentChoice.select("HR");
    statusChoice.select("Active");
}
}

```

## Main.Java

```

public class Main {

    public static void main(String[] args) {

        EmployeeCRUD employeeCRUD =
            new EmployeeCRUD();

        employeeCRUD.setVisible(true);
    }
}

```

OUTPUTS:

ADD:

Employee Management and Payroll System

EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

Employee ID \*

101

Name \*

sripathi

Gender \*

Male

Date of Birth \*

2005-08-19

Contact Number \*

34131313

Email

sripathi.maram25@gmail.com

Address

ondddad

Department \*

HR

Designation \*

dadad

Date of Joining \*

2024-08-01

Basic Salary \*

131313

Allowances

200

Deductions

212

Employment Status \*

Active

Search By: Employee ID

Add

Search

Update

Delete

Clear

View All

Payroll

Employee Added Successfully

37 • select \* from Employee;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

|   | employee_id | name     | gender | dob        | contact  | email                      | address | department_id | designation | joining_date | basic_salary | al   |
|---|-------------|----------|--------|------------|----------|----------------------------|---------|---------------|-------------|--------------|--------------|------|
| ▶ | 101         | sripathi | Male   | 2005-08-19 | 34131313 | sripathi.maram25@gmail.com | ondddad | 1             | dadad       | 2024-08-01   | 131313       | 20   |
| * | NULL        | NULL     | NULL   | NULL       | NULL     | NULL                       | NULL    | NULL          | NULL        | NULL         | NULL         | NULL |

Search:

Employee Management and Payroll System

EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

Employee ID \*

101

Name \*

sripathi

Gender \*

Male

Date of Birth \*

2005-08-19

Contact Number \*

34131313

Email

sripathi.maram25@gmail.com

Address

ondddad

Department \*

HR

Designation \*

dadad

Date of Joining \*

2024-08-01

Basic Salary \*

131313.0

Allowances

200.0

Deductions

212.0

Employment Status \*

Active

Search By: Employee ID

101

Add

Search

Update

Delete

Clear

View All

Payroll

Employee Found Successfully

## UPDATE:

Employee Management and Payroll System

Employee ID \*

101

Name \*

Maram

Gender \*

Male

Date of Birth \*

2005-08-19

Contact Number \*

4522442

Email

sripathi.maram25@gmail.com

Address

ssflsdf

Department \*

HR

Designation \*

dsgsfd

Date of Joining \*

2005-08-01

Basic Salary \*

3114133

Allowances

5000

Deductions

300

Employment Status \*

Active

Search By: Employee ID

Add

Search

Update

Delete

Clear

View All

Payroll

Employee Updated Successfully

## VIEW ALL

Employee Management and Payroll System

Employee ID \*

Name \*

Gender \*

Male

Date of Birth \*

Contact Number \*

Email

Address

Department \*

HR

Designation \*

Date of Joining \*

Basic Salary \*

Allowances

Deductions

Employment Status \*

Active

Search By: Employee ID

Add

Search

Update

Delete

Clear

View All

Payroll

ID: 101 Name: Maram Department: HR Designation: dsgsfd Status: Active Basic Salary: 3114133.0

## Payroll

Employee Management and Payroll System

EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

Employee ID \*

Gender \*

Male

Contact Number \*

Address

Designation \*

Basic Salary \*

20000

Deductions

Name \*

Date of Birth \*

Email

Department \*

HR

Date of Joining \*

Allowances

Employment Status \*

Active

Search By: Employee ID

Add

Search

Update

Delete

Clear

View All

Payroll

PAYROLL DETAILS

Basic Salary : 20000.0  
Allowances : 0.0  
Deductions : 0.0  
Gross Salary : 20000.0  
Net Salary : 20000.0

Done By  
Kalyan Reddy  
192472274